May 31, 2025

`LcvStall` Bus Stuff

1. Changes to
2. **I won't be making a crossbar switch yet**. Also, **at a later date**, I might just abandon my own bus for a non-custom one (like TileLink)
3. Type Generics:
   1. `sendData`: Data Type for stuff sent from Host to Device
      - Must include `data`, `addr`, `byteEn`, `burstSize`, and `isWrite` **at least**.
      - `burstSize` is in units of whatever width `data` is.
        - `burstSize` of `0x0` still means `0x0` units
   2. `recvData`: Data Type for stuff sent from Device to Host
      - Must include `data` **at least**.
4. `LcvStallSlicer`:
   1. For this test (and also in general), I wish to use an `LcvStall` "Address Space Slicer" module of some sort.
   2. This will (combinatorially?) slice up an address space using some of the high bits of an address to select an `LcvStall` bus device to access.
      - I think this **can be done combinatorially** because wires/slicing is free, for some definition of "free" anyway. FPGA routing issues come to mind….
      - Later I might also support doing this in a **registered** manner. Or alternatively, the bus devices and bus host attached to could just do that themselves?
   3. Ports:
      1. A single `LcvStall` bus host.
      2. Multiple `LcvStall` bus devices.
5. `LcvStallArbiter`:
   1. This module will arbitrate access from multiple bus hosts to a single bus device.
   2. Two modes of arbitration (probably selected via generics, but maybe could be determined at FPGA "runtime"?):
      1. Round-Robin Arbitration:
         - Simple method of implementation (possibly the one I go with?) is to use a counter that is used to index into the bus. This could be slower in terms of cycles, but it *may* be faster for the
      2. Priority Arbitration:
6. Combinining `LcvStallSlicer` and `LcvStallArbiter` should, *at least for this project*, eliminate the need for a crossbar switch, methinks.